信息学竞赛培训 · C++ 程序设计

LESSON 05 / DIVIDE & CONQUER

Binary Search

分治算法（一）：二分查找

05

整数域模板 · 实数域模板 · 二分答案 · P2249 查找精讲

> 上一课我们把所有可能试了一遍。这一课换个思路：只要数据有序，每次都能理直气壮地扔掉一半——log 次之后，答案自己浮出来。

授课班级
中预 2026 级 B 班

主讲
信息教研组

共 5 道经典例题 · 含洛谷题号

05

学习目标与课堂安排

第 5 课 · 分治算法（一）二分查找

本节课你将学会

1 背熟整数域二分模板

闭区间 while (l <= r)：取中点、比大小、左右各缩一半，三个分支一个都不能错。

2 掌握实数域二分的写法

不定精度定次数：固定迭代 100 次，范围必然缩到 10^{-30} 量级，比调 eps 更稳。

3 会写「二分答案」

答案有单调性、检验只要 O(n) 时，就别枚举答案——二分它。P1873、P3743、P1163 都是这一招。

4 有「先问单调性」的习惯

二分不是万能钥匙：数据无序就先排序，问题无单调性就直接放弃这条路。

课堂安排

15 分钟	概念与模板
25 分钟	精讲 T01 P2249
55 分钟	上机实践（5 题）
10 分钟	总结答疑

承上启下

第 3 课是「全部试一遍」，第 4 课教会我们排序。这一课把两件事接上：排好序，是为了让二分有资格出场。

什么是二分查找？

一句话定义

在有序的数据里找答案：每次看中间那个，比一下大小，就能一次性排除掉一半不可能的位置——重复到区间里只剩一个数。

生活里的一次二分 —— 猜数字（1 ~ 1000）

心里想一个数 → 先猜 500 → 太大了？剩下 1~499 → 最多 10 次必中

① 有序

1~1000 默认从小到大。数据乱序，二分无从下手。

② 中间

永远问区间正中的那个数，信息量最大。

③ 砍半

一次比较只留一边，范围严格减半。

④ 收敛

10 次 = $\log_2$ 1000，答案是"必中"而非"运气好"。

暴力查找要看 1000 次，二分只要 10 次。数据越大，差距越夸张：10 万条数据，暴力 10 万次，二分 17 次。

二分的四步：定区间 · 取中 · 比较 · 缩半

① 定区间

l = 0，r = n − 1（闭区间）。答案一定在 [l, r] 之内——这是整段代码的契约。

② 取中

mid = l + (r − l) / 2。整数除法向下取整，l 与 r 相邻时 mid 落在左边。

③ 比较

拿 a[mid] 与目标比：相等就收工；目标更大丢左半；目标更小丢右半。

④ 缩半

l = mid + 1 或 r = mid − 1——必须带上 ±1，否则区间可能原地不动，死循环。

区间是怎么缩的（目标 = 7）

[0, 7]	1 3 5 7 9 11 13 15　mid=3 → 7 命中
[0, 3]	目标 5：mid=1 → a[1]=3 太小 → 留右半
[2, 3]	只剩 2 个数，再取中即收敛

每轮区间长度大约减半：8 → 4 → 2 → 1。n = 8 只要 3 次，n = 10^5　只要 17 次。

一句话记住

二分的全部智慧就在"砍掉一半"这六个字——其余都是为它服务的细节。

区间缩展示意：三个指针怎么走

数组 a = [1, 3, 5, 7, 9, 11, 13, 15]，下标 0~7。要在里面找 目标 = 11。

第 1 轮　　l = 0　　r = 7　　mid = 3



a[3] = 7 < 11 → 目标在右半 → l = mid + 1 = 4

第 2 轮　　l = 4　　r = 7　　mid = 5



a[5] = 11 = 目标 → 找到，返回下标 5 ✓

灰虚线 = 已经排除

被扔掉的部分永久排除，不会再看第二眼。

深蓝 = 当前比较

每轮只比较 1 次，其余位置连碰都不碰。

2 轮解决 8 个数

暴力最多 8 次，二分 3 次封顶（这里是运气好）。

整数域二分模板：闭区间写法

模板：在有序数组 a 中查找 x

```
// 闭区间 [l, r]：答案限定在这个范围内
int l = 0, r = n - 1;
int ans = -1;                  // 没找到就保持 -1

while (l <= r) {               // 区间非空就继续
    int mid = l + (r - l) / 2;
    if (a[mid] == x) {
        ans = mid;             // 找到，记下来
        break;                 // 只要任意一个下标
    } else if (a[mid] < x) {
        l = mid + 1;           // 左半全部 < x，扔掉
    } else {
        r = mid - 1;           // 右半全部 > x，扔掉
    }
}
// 循环结束后：ans 为答案，-1 表示不存在
```

为什么是 l <= r

闭区间里 l = r 时还剩 1 个元素，必须再比一次才能确认。写成 l < r 会漏掉最后一个数。

为什么 ±1 不能省

a[mid] 已经比过了，mid 本身可以排除。写成 l = mid 会原地打转，直接死循环。

前提：数组必须有序

"左半都小于 x" 这个判断，只在有序时成立。无序先 sort（第 4 课刚学过）。

复杂度

O(log n)：10 万数据最多 17 轮，100 万也才 20 轮。

陷阱：mid 该怎么算？

不推荐 `int mid = (l + r) / 2;`

l 和 r 都是 int，n 接近 2×10^9 时 l + r 会超过 int 上限（约 2.1×10^9），直接溢出成负数 → 下标越界或死循环。

小数据永远测不出来，大数据一交就 RE。

推荐 `int mid = l + (r - l) / 2;`

先算「区间长度」再折半，r − l 永远不超过区间本身，不会溢出。同时它也明确表达了「从 l 出发走一半」的语义。

写熟这一种，以后不用再想。

两个必须记住的取整细节

整数除法是向下取整

(5 − 4) / 2 = 0，所以 l = 4、r = 5 时 mid = 4，落在左中位。

「取左中位」是好事

因为 mid 永远 < r，配合 l = mid + 1，区间一定在缩小，不会卡住。

写「取右中位」要小心

l + (r − l + 1) / 2 时，若左边界写 l = mid 就会死循环，必须配 r = mid − 1。

统一约定：l 从 0 开始、r 取 n − 1、mid 用 l + (r − l) / 2、分支写 ±1——这四句话背下来，二分的框架就稳了。

三种变体：找值 · 找下界 · 找上界

数组 a = [1, 3, 3, 3, 5, 7]，目标 x = 3 —— 同一个数组，三种问法给出三个不同答案。

① 找值

存在性 / 任意一个下标

```
相等即 break
返回 ans / -1
```

答案可能是 1、2、3

任意一个都对——题目只要"有没有"，就够用。

② 找下界（第一个 ≥ x）

等于 x 的最左位置

```
a[mid] >= x 时
ans = mid; r = mid - 1;
```

答案唯一：下标 1

相等时不 break，继续往左找更早的。

③ 找上界（第一个 > x）

等于 x 的最右位置 + 1

```
a[mid] > x 时
ans = mid; r = mid - 1;
```

答案唯一：下标 4

只问"第一个严格大于"，不关心等于。

最有用的组合：个数 = 上界 − 下界

想知道 3 出现了几次？找第一个 3（下标 1）和第一个 > 3 的位置（下标 4），4 − 1 = 3 次。T02 A-B 数对正是靠这一招。

三个模板骨架完全一样，只是「相等时怎么办」不同——找值 break，找下界往左，找上界往右。

实数域二分模板：不定精度，定次数

模板：求满足 check(x) 的最大 x

```
// 实数域：不定 eps，固定迭代 100 次
double l = 0.0, r = 1e9;    // 答案所在范围
for (int it = 0; it < 100; it++) {
    double mid = (l + r) / 2.0;
    if (check(mid)) {
        l = mid;            // mid 可行 → 答案在右半
    } else {
        r = mid;            // mid 不可行 → 答案在左半
    }
}
// 结束后 l 与 r 已贴合，输出 l 即可
printf("%.10f\n", l);
```

为什么不用 while (r - l > 1e-6)

eps 选大了精度不够，选小了像 1e-12 在 double 下可能永远达不到 → 死循环。固定次数永远不会出事。

100 次够不够？

每次减半，100 次 = 除以 $2^{100} \approx 10^{30}$。范围 10^9 也会缩到 10^{-21}，远超 double 的 15 位有效数字。

为什么没有 ±1

实数连续，mid 就是答案本身，不存在"排除掉 mid"的问题，直接令 l = mid 或 r = mid。

输出建议

题目常要求 10 位小数，用 printf("%.10f")，比 cout 更可控。

整数域 vs 实数域：一张表说清差异

模板一 + 模板二对照

对比项	整数域二分	实数域二分
区间写法	闭区间 [l, r]	连续区间 [l, r]
循环条件	while (l <= r)	for 100 轮（固定次数）
中点	l + (r − l) / 2	(l + r) / 2.0
边界移动	l = mid + 1 / r = mid − 1	l = mid / r = mid
结束状态	l > r，区间为空	l 与 r 贴合到 10^{-20}
答案在哪	ans 变量 or 最后 l / r	直接输出 l 或 r
典型用途	下标、次数、排名	比例、时间、长度
本课例题	T01 / T02 / T03	T04 / T05

共同的灵魂：单调性

两套模板只是外壳。真正决定"能不能二分"的，是 check(x) 随 x 单调变化——这一点两域完全一致。

最容易混的两处

① 整数域忘了 ±1 → 死循环；② 实数域照抄 ±1 → 精度莫名变差（把可能的答案排除了）。

判断口诀

答案能不能"一个一个数"？能数 → 整数域；只能"连续地靠近" → 实数域。

本课五题总览：从模板到实战

第 5 课 · 分治算法（一）二分查找

从精讲到自主练习

编号	题目（洛谷题号）	来源	难度	二分形态与关键动作
T01	**查找 · P2249**	洛谷 · 深基13 例1	普及−	整数域找下界：相等时不 break，继续往左缩
T02	**A-B 数对 · P1102**	洛谷经典	普及−	排序 + 两次二分：个数 = 上界 − 下界
T03	**砍树 · P1873**	COCI 2011/2012	普及−	二分答案（整数）：h 越大木材越少
T04	**小鸟的设备 · P3743**	洛谷经典	普及	二分答案（实数）：check 判断总缺口
T05	**银行贷款 · P1163**	洛谷经典	普及−	二分答案（实数）+ 复利公式

节奏　T01 精讲 4 页；T02－T05 各 4 页（题面 → 思路 → 手推 → 代码）。T03 起进入"二分答案"，这是本课的重点跃迁。

完成度　T01－T02 课上必做（模板熟练）；T03 争取当堂拿下；T04－T05 若课时紧可留作课后挑战。

精讲 T01 | 查找（P2249）

第 5 课 · 分治算法（一）二分查找

洛谷 · 深基13 例1 · 难度 ★★

题面描述

输入 n 个不超过 10^9 的单调不减（后面的数不小于前面的数）非负整数 $a_1 \cdots a_n$，然后进行 m 次询问。对于每次询问，给出一个整数 q，要求输出这个数字在序列中第一次出现的编号，如果没有找到则输出 -1。

三个不能漏读的字

「单调不减」= 二分资格；「第一次出现」= 找下界不是找值；「编号从 1 开始」= 输出要 +1。

输入格式

第一行 n 和 m；第二行 n 个数；第三行 m 个询问值。

输出格式

一行 m 个整数，空格隔开；不存在输出 -1。

数据范围

$n \leqslant 10^6$，$m \leqslant 10^5$，数值 $\leqslant 10^9$。输入输出量大，需要快速读入。

样例

输入

```
11 3
1 3 3 3 5 7 9 11
13 15 15
1 3 6
```

输出

```
1 2 -1
```

3 第一次出现在第 2 位；6 不存在。

思路预告

数组已经排好序了，题目还一次问 m 个——这正是二分的标准场景。关键是把"第一个"翻译成"下界"。

思路梳理：把「第一次出现」翻译成「下界」

① 直接查值为什么不行

用最基础的「相等就 break」模板，mid 一旦撞上 3 就立刻返回——但撞到的是第 2 个还是第 3 个 3，完全看运气。

② 改成「下界」语义

我们要的是「第一个 ⩾ q 的位置」。a[mid] ⩾ q 时，mid 本身可能就是答案，但左边也许还有——记下 mid，然后 r = mid − 1 继续往左找。

③ 收尾判存在

循环结束后检查 a[ans] 是否真的等于 q。若第一个 ⩾ q 的元素比 q 大，说明 q 根本不存在 → 输出 −1。

一句话记住

「第一个出现」= 「下界」= 相等也不停，继续往左压。

下界模板（与第 6 页的差异只在相等分支）

```
int l = 0, r = n - 1, ans = -1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    if (a[mid] >= q) {   // 可能还能更左
        ans = mid;       // 先记下候选
        r = mid - 1;     // 继续往左
    } else {
        l = mid + 1;     // 左半全 < q
    }
}
if (ans != -1 && a[ans] == q)
    printf("%d ", ans + 1);  // 编号从 1 开始
else
    printf("-1 ");
```

若 q 比数组里所有数都大，ans 会停在 −1；若 q 落在两个数之间，a[ans] ≠ q 也要判 −1。

手推一遍：q = 3 时，l / r / mid 怎么走

数组	1	2	3	4	5	6	7	8	9	10	11
	1	3	3	3	5	7	9	11	13	15	15

轮次	l	r	mid	a[mid]	判定	动作
1	1	11	6	7	7 ⩾ 3，右边全不要	ans = 6，r = 5
2	1	5	3	3	3 ⩾ 3，左边可能还有	ans = 3，r = 2
3	1	2	1	1	1 < 3，左半全无效	l = 2
4	2	2	2	3	3 ⩾ 3，再往左试	ans = 2，r = 1
结束	2	1	—	—	l > r，循环退出	输出 ans = 2

第 2 轮是真正的关键

mid = 3 时 a[mid] 已经等于 q 了，但下界语义要求「相等也不停」，必须再往左压一次，才拿到第 2 个位置。

只用了 4 次比较

n = 11，逐个扫要 11 次；二分 $\lceil \log_2 11 \rceil + 1 = 5$ 次封顶。n 越大差距越夸张。

参考代码：一次预处理，m 次查询全变 O(log n)


```cpp
#include <cstdio>
const int MAXN = 1000005;
int a[MAXN];

int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    while (m--) {
        int q; scanf("%d", &q);
        int l = 0, r = n - 1, ans = -1;
        while (l <= r) {
            int mid = l + (r - l) / 2;
            if (a[mid] >= q) { ans = mid; r = mid - 1; }
            else             l = mid + 1;
        }
        if (ans != -1 && a[ans] == q) printf("%d ", ans + 1);
        else                          printf("-1 ");
    }
    printf("\n");
    return 0;
}
```

坑 1：输出用 ans 还是 ans + 1

题目编号从 1 起，数组下标从 0 起。二分全程用 0 基下标，只在最后输出时 +1。

坑 2：忘了判 a[ans] == q

ans 只是「第一个 ⩾ q 的位置」，q 不存在时它指向一个更大的数，必须回头验一遍。

坑 3：mid 写成 (l + r) / 2

n 上到 10^6 时 l + r 可能超过 int 范围变负数，用 l + (r − l) / 2 才稳。

复杂度

预处理读入 O(n)，单次查询 O(log n)，总计 O(n + m log n)。暴力 O(nm) 在 10^5 × 10^5 下必 TLE。

例 T02：A − B 数对（P1102）

普及− · 哈希 / 二分 · 数学变形

题目描述

给出一串正整数数列以及一个正整数 C，要求计算出所有满足 A − B = C 的数对的个数（不同位置的数字一样的数对算不同的数对）。

例如对数列 1 1 2 3、C = 2：满足条件的数对有 （1, 3）、（1, 3）、（2, 3），共 3 个。

输入格式

第一行两个正整数 N 和 C。第二行 N 个正整数，每个数不超过 2^{30} 。

输出格式

一行，表示满足条件的数对个数。

样例

输入

```
4 2
1 1 2 3
```

输出

```
3
```

数据规模（决定算法）

$N \leqslant 2 \times 10^5$ ，数值 $\leqslant 2^{30}$ $\approx 1.07 \times 10^9$ 。

双循环枚举所有对是 $O(N^2) \approx 4 \times 10^{10}$ —— 必超时。

答案最大接近 $N^2 / 4$，约 10^{10} —— 必须开 **long long**。

先想一想

把 A − B = C 变成 A = B + C。固定一个 B，是不是就只剩「查 C 个数在不在、有几个」的问题了？

核心变形：A = B + C，问题变成「数个数」

$$A - B = C \iff A = B + C$$

枚举 B，只要知道「B + C 在数组里出现了几次」

① 为什么排序后能数个数

升序排列后，所有等于某个值的元素必然连成一段连续区间。这段区间的长度，就是这个值出现的次数——而区间两端，正好是下界和上界。

② 两次二分求区间长度

lower(v) = 第一个 ⩾ v 的位置；upper(v) = 第一个 > v 的位置。二者相减就是 v 的个数，无需额外判断。

③ 总复杂度

排序 O(N log N)，每个元素做两次二分 O(log N)，总计 O(N log N)，$N = 2 \times 10^5$ 完全够用。

以样例 1 1 2 3（去重后 1 2 3）举例

B	B + C	lower	upper	个数
1	3	3	4	1
1	3	3	4	1
2	4	4	4	0
3	5	4	4	0

两个 1 各贡献一次，合计 3，与样例一致。

易错：重复元素不能去重

题目明确「不同位置的数字一样的数对算不同数对」，所以不能先 unique。正是因为有重复，才需要「数个数」而不是「判存在」。

两个小函数：只差一个等号，语义差一整段

lower(v)：第一个 ⩾ v 的下标

```
int lower(int v) {
    int l = 0, r = n - 1, ans = n;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (a[mid] >= v) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    return ans;   // 全小于 v 时返回 n
}
```

upper(v)：第一个 > v 的下标

```
int upper(int v) {
    int l = 0, r = n - 1, ans = n;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (a[mid] > v) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    return ans;   // 全不大于 v 时返回 n
}
```

ans 的初值为什么是 n 而不是 −1

用 n 表示「不存在的空位置」。这样 upper − lower 天然为 0，不必写 if 特判。

数个数 = upper(v) − lower(v)

以 1 1 2 3 为例，lower(3) = 3、upper(3) = 4，相减得 1，恰好是 3 的个数。

只改一个符号

把 **>=** 改成 **>**，其余完全不动——这正是第 8 页说的「两种变体」。

T02 参考代码：排序 + 每个元素两次二分

两个边界函数

```cpp
int lower(int v) {
    int l = 0, r = n - 1, ans = n;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (a[mid] >= v) { ans = mid; r = mid - 1; }
        else             l = mid + 1;
    }
    return ans;
}
int upper(int v) {
    int l = 0, r = n - 1, ans = n;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (a[mid] > v) { ans = mid; r = mid - 1; }
        else            l = mid + 1;
    }
    return ans;
}
```

主函数：枚举 B，数 B + C 的个数

```cpp
int main() {
    scanf("%d%d", &n, &c);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    sort(a, a + n);

    long long ans = 0;
    for (int i = 0; i < n; i++) {
        long long v = (long long)a[i] + c;
        ans += upper(v) - lower(v);
    }
    printf("%lld\n", ans);
    return 0;
}
```

复杂度

排序 O(N log N) + 2N 次二分 O(N log N)，总计 O(N log N)。

坑 1：ans 不开 long long

答案可达 10^{10} ，int 上限约 2.1×10^9 ，会溢出成负数。

坑 2：a[i] + c 溢出

两个 int 相加先按 int 算，先转 long long 再相加。

坑 3：重复计数

只查 B + C，别同时查 B − C，否则每对算两遍。

另一种解法

用 map / 手写哈希记录每个值出现次数，枚举 B 直接查表，O(N) 期望时间。

例 T03：砍树（P1873）

题目描述

伐木工米尔科需要砍下 M 米长的木材。他有一排 N 棵树，第 i 棵树高 H_i。

他设定一个高度 H，把每棵高出 H 的部分切下来（不高于 H 的树不产生木材）。为了环保，H 尽可能大。求最大的 H。

输入格式

第一行两个整数 N、M。第二行 N 个整数表示树高。

输出格式

一个整数，表示最大的 H。

样例

输入

```
4 7
20 15 10 17
```

输出

```
15
```

H = 15 时得到 (20−15) + 0 + 0 + (17−15) = 7 米，正好；H = 16 只有 5 米，不够。

数据规模

$N \leqslant 10^6$ ，树高 $\leqslant 10^9$ ，$M \leqslant 2 \times 10^9$ 。

H 的取值从 0 到 10^9 ，逐个试是 10^9 次 × 10^6 棵树 —— 绝不可能。

注意

答案不是「数组里的某个值」，而是一个被人为选定的高度 —— 这一类和「查数组中元素」的二分完全不同。

二分答案：把「求最优」变成「判可行」

① 单调性：H 越大，砍得越少

把 H 设得越高，每棵树切下来的部分就越短。所以「能砍够 M 米」这件事对 H 是单调的：存在一个分界点，左边都够、右边都不够。

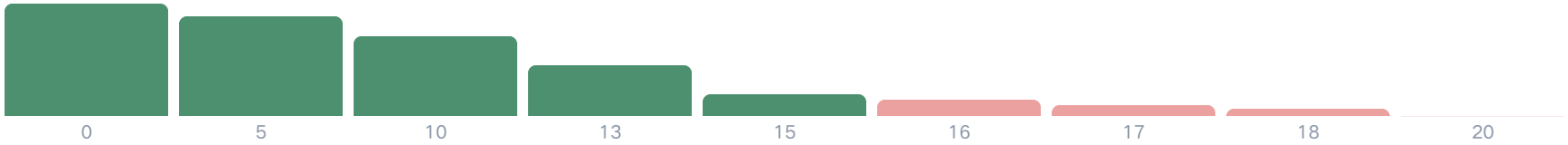
② 二分答案：在 H 的取值域上二分

二分对象不再是数组，而是答案的可能取值 [0, maxH]。每次取 mid，判它够不够；够就往高处试，不够就往低处退。

③ check：O(N) 暴力算一遍

给定 H，把所有 h[i] > H 的差值累加，与 M 比较。单次 O(N)，乘上二分 30 轮，总计 3×10^7 ，可过。

单调性示意图（横轴 H，纵轴砍到的木材量）



绿色：check 为真（≥ M）

红色：check 为假（< M）

分界点在 15 与 16 之间，最大可行值就是 15 —— 二分要做的事，就是把这个 15 找出来。

check 函数怎么写：两件事，一个返回值

check(H)：H 这个高度能不能砍够 M 米

```
bool check(long long h) {
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        if (tree[i] > h) {
            sum += tree[i] - h;
        }
        if (sum >= m) return true;  // 提前退出
    }
    return sum >= m;
}
```

写 check 的三条纪律

1. 只用「可行 / 不可行」两个答案，不夹带其他信息；
2. 累加量一律开 long long，N 棵树 × 10^9 会到 10^{15} ；
3. 能在中途判定就提前 return，常数小一半。

二分区间怎么定：l = 0，r = max(tree)

H 取 0 一定可行（砍下全部树高，远多于 M）；H 取最大树高时木材为 0，若 M > 0 一定不可行。所以答案必在 [0, maxH] 内。

写模板时用「可行就记 ans、往高处试」的形式，与第 8 页的下界变体同构。

样例手推：N = 4，M = 7，树高 20 15 10 17

l	r	mid	砍到的量	check	动作
0	20	10	10 + 5 + 0 + 7 = 22	真	ans = 10，l = 11
11	20	15	5 + 0 + 0 + 2 = 7	真	ans = 15，l = 16
16	20	18	2 + 0 + 0 + 0 = 2	假	r = 17
16	17	16	4 + 0 + 0 + 1 = 5	假	r = 15，退出

循环退出时 ans 停在 15，与样例输出一致。

T03 参考代码：二分答案的标准骨架


```cpp
#include <cstdio>
const int MAXN = 1000005;
int tree[MAXN], n, m;

bool check(long long h) {
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        if (tree[i] > h) sum += tree[i] - h;
        if (sum >= m) return true;
    }
    return false;
}

int main() {
    scanf("%d%d", &n, &m);
    int maxH = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &tree[i]);
        if (tree[i] > maxH) maxH = tree[i];
    }

    int l = 0, r = maxH, ans = 0;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (check(mid)) { ans = mid; l = mid + 1; }
        else              r = mid - 1;
    }
    printf("%d\n", ans);
    return 0;
}
```

骨架背下来，换题只换 check

定边界 → while(l ⩽ r) → 求 mid → check 决定记不记 ans、往哪边挪。

坑 1：可行时往左挪

本题求最大 H，可行就要往高处试（l = mid + 1）。方向写反，答案会停在 0。

坑 2：check 里用 int 累加

$10^6 \times 10^9 = 10^{15}$，远超 int。sum 必须 long long。

复杂度

O(N log(maxH)) ≈ $10^6 \times 30 = 3 \times 10^7$，实测约 0.1 秒。

例 T04：小鸟的设备（P3743）

题目描述

有 n 个设备，第 i 个设备初始电量为 b_i，每秒耗电 a_i。你有一个充电宝，可以向任意一个设备以每秒 1 单位的速度充电（同一时刻只能给一个设备充电）。

求最多能让所有设备同时运行多少秒？如果永远能运行，输出 −1。

输出要求

保留到小数点后 10 位（题目会给误差容忍，一般 10^{-6} 即可）。

样例

输入

```
2 1
3 2
4 2
```

输出

```
2.0000000000
```

这题为什么不能用整数二分

答案是「秒数」，可以是 2.71828… 这样的小数。整型 mid 会把答案粗化到整数，精度直接错。

check(t)：先算总缺口，再看总量够不够

① 逐台算缺口

第 i 台跑 t 秒要 $a_i \cdot t$，自带 b_i。不够的部分 $\max(0, a_i \cdot t - b_i)$ 必须由充电宝补。

② 缺口总量 ⩽ 充电量

充电宝 t 秒只能充 t 单位（同一时刻只充一台），所以总缺口不超过 t 才可行。

③ 单调性

t 增大时每个缺口都单调不减，总量自然单调不减，而补给上限 t 只是线性增长 —— 存在唯一分界点。

$$\text{check}(t)：\sum \max(0, a_i \cdot t - b_i) \leqslant t$$

$\sum a_i \leqslant 1$ 时缺口增速不超过补给速度，永远够 → 输出 −1

样例验算：a = 3 2，b = 4 2

t = 1.5：需求 4.5 和 3，缺口分别为 0.5 和 1，合计 1.5；补给上限也是 1.5，正好够 —— 可行。

t 再大一点，缺口增长快于补给，立刻不可行。所以最大可行值就在 1.5 附近。

为什么这题只考「二分框架」

本题真正的难点在 check 的数学建模，二分区间的写法与整数域完全一样。课堂上重点看：实数域怎么定循环、怎么定边界。

实数域二分：为什么固定跑 100 轮

固定轮数写法（推荐）

```
double l = 0.0, r = 1e10;
for (int it = 0; it < 100; it++) {
    double mid = (l + r) / 2.0;
    if (check(mid)) l = mid;   // 可行，往大处试
    else            r = mid;   // 不可行，往小处退
}
printf("%.10f\n", l);
```

不推荐的写法：while (r − l > 1e-9)

当区间起点是 0、终点是 10^{10} 时，要缩到 10^{-9} 需要约 64 轮；但若 eps 写得更小，或浮点精度已经到极限，条件可能永远不满足 —— 死循环。

100 轮能到什么精度

轮数	区间长度	含义
1	5×10^{9}	刚砍一半
10	约 10^{7}	误差 1 千万
30	约 10	误差 10 秒
50	约 10^{-5}	远超题目要求
100	约 10^{-20}	已小于 double 精度

每轮区间减半，100 轮后长度是初始的 2^{-100}，远小于 double 的 15~16 位有效数字 —— 再迭代也不会更准，只会更慢。

两条铁律

① l 和 r 是 double，mid 不要写成 int；

② 输出用 l 而不是 r —— l 始终停在「可行」一侧，更保守。

T04 参考代码：先判无解，再实数二分


```cpp
#include <cstdio>
const int MAXN = 100005;
int n, a[MAXN];
double b[MAXN];

bool check(double t) {
    double need = 0.0;
    for (int i = 0; i < n; i++) {
        double use = (double)a[i] * t;
        if (use > b[i]) need += use - b[i];
        if (need > t) return false;
    }
    return need <= t + 1e-9;
}

int main() {
    scanf("%d%*d", &n);
    double suma = 0.0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]); suma += a[i];
    }
    for (int i = 0; i < n; i++) scanf("%lf", &b[i]);
    if (suma <= 1.0 + 1e-9) { printf("-1\n"); return 0; }

    double l = 0.0, r = 1e10;
    for (int it = 0; it < 100; it++) {
        double mid = (l + r) / 2.0;
        if (check(mid)) l = mid;
        else            r = mid;
    }
    printf("%.10f\n", l);
    return 0;
}
```

坑 1：不判 Σa ⩽ 1 直接二分

永远可行时上界取多大都「可行」，二分结果就是上界 10^{10} ，而非 −1。

坑 2：check 里用 int 算 a[i] * t

先把 a[i] 转 double，否则中间结果被截断成整数，精度全丢。

坑 3：上界取小了

b_i 可达 10^9 、a_i 最小 1，最长能撑约 10^9 秒，上界至少取 10^9 以安全，这里直接 10^{10} 。

读入顺序要看清

本题输入是「n、k」后跟 a 数组、再跟 b 数组，两行分开给，别按「a b」成对读。

例 T05：银行贷款（P1163）

第 5 课 · 分治算法（一）二分查找

普及− · 实数二分 + 复利模拟

题目描述

当一个人银行贷款，要每月还固定的金额。现在给出贷款额、每月还款额和还清所需月数，求月利率（按复利计算）。

假设月利率为 r，则每月还完后剩余欠款按 (1 + r) 计息。要求输出月利率，保留一位小数（百分数形式）。

输入 / 输出

一行三个浮点数：贷款额、月还款额、月数。输出一个月利率，以百分数表示，保留 1 位小数。

样例

输入

```
1000 100 12
```

输出

```
2.9
```

借 1000 元，每月还 100 元，12 个月还清 → 月利率约 2.9%。

这题的「二分对象」是什么

月利率 r。r 越大，还清越快；给定月数和还款额，r 是唯一确定的 —— 用二分把它试出来。

复利递推：一行公式模拟 n 个月

欠款 ← (欠款 − 月还款) × (1 + r)

每月先还钱，剩下的再滚一个月利息，重复 n 次

check(r)：按这个利率，n 个月后是否还不清

```
bool check(double r) {
    double debt = loan;          // 贷款额
    for (int i = 0; i < months; i++) {
        debt = debt * (1 + r) - pay;
        if (debt <= 0) return false;  // 提前还清
    }
    return debt > 0;             // 还有欠款 = 利率偏低
}
```

check 返回的是「利率太低」

这与前几题的「可行」语义相反，写的时候务必在函数头注释清楚，避免二分方向搞反。

r 越大，剩余欠款越少

r（月利率）	12 个月后欠款	判断
1%	约 120 元	还不清，利率偏低
2%	约 43 元	还不清，利率偏低
2.9%	约 0.4 元	接近临界
3.5%	约 −38 元	提前还清，利率偏高
5%	约 −130 元	提前还清，利率偏高

欠款随 r 单调递减，所以「剩余欠款 > 0」是一段前缀 —— 又是一个单调分界点。

二分区间的经验值

月利率一般不会超过 100%（现实中年化几十就离谱了），取 [0, 5] 已经足够宽裕。上界给宽一点不影响结果，只多几轮迭代。

T05 参考代码：二分利率，输出百分数


```cpp
#include <cstdio>
double loan, pay;
int months;

bool stillOwe(double r) {
    double debt = loan;
    for (int i = 0; i < months; i++) {
        debt = debt * (1 + r) - pay;
        if (debt <= 0) return false;
    }
    return debt > 0;
}

int main() {
    scanf("%lf%lf%d", &loan, &pay, &months);

    double l = 0.0, r = 5.0;
    for (int it = 0; it < 100; it++) {
        double mid = (l + r) / 2.0;
        if (stillOwe(mid)) l = mid;   // 还不清，利率偏低
        else               r = mid;   // 已还清，利率偏高
    }
    printf("%.1f\n", l * 100.0);
    return 0;
}
```

为什么输出 l × 100

二分的对象是小数利率 r。题目要百分数，所以乘 100。l 停在「还不清」一侧，取它偏保守。

坑 1：months 用 double 读

月数是整数，用 %d 读。混用 %lf 会导致后续读入错位。

坑 2：二分方向写反

stillOwe 为真说明利率不够，要往高处试（l = mid）；写反会收敛到 0。

坑 3：函数名误导

叫 stillOwe 比叫 check 更清楚 —— 能让人一眼看出返回值含义，减少方向错误。

复杂度

O(100 × months)，months 一般不超过几百，可以忽略。

五道题串起来看：二分到底在找什么

题号	二分对象	check / 判定	关键差异
T01 P2249 查找	数组下标	a[mid] ⩾ q	相等不停，找下界
T02 P1102 A−B 数对	数组下标	两次二分相减	数个数而非判存在
T03 P1873 砍树	答案 H（整数）	$\Sigma(h_i - H) \geqslant M$	在答案域上二分
T04 P3743 小鸟设备	答案 t（实数）	Σ缺口 ⩽ t	实数域，固定 100 轮
T05 P1163 银行贷款	答案 r（实数）	n 月后仍欠款	check 语义是「偏小」

前两类：在「已知数据」里找

T01、T02 的二分对象是数组下标，前提是数组已排序。这类题的重点在边界语义：找下界还是找上界、相等时往哪边走。

后三类：在「答案空间」里找

T03、T04、T05 的二分对象是答案本身。这类题的重点全在 check：先证明单调性，再把「满足条件」写成 O(n) 的判定。

判断依据只有一句话

答案是不是数组里现成的元素？是 → 下标二分；不是 → 答案二分。这一句能帮你一秒分辨题型。

错误门诊：二分题的四种典型症状

① 死循环（TLE 且无输出）

症状：程序卡死。病灶：l = mid 或 r = mid 却没动 mid 的取值。

对策：整数域永远 l = mid + 1 / r = mid − 1；实数域才用 l = mid（因为有固定轮数兜底）。

② 答案差 1（WA 在边界点）

症状：大部分点对，个别点差 1。病灶：区间开闭用混了，或 ans 初值没设对。

对策：全程统一用闭区间 [l, r]、ans 设为不可能的哨兵（如 n 或 −1）。

③ 溢出（大负数或全错）

症状：小数据对、大数据错，答案变负数。病灶：mid = (l + r) / 2 或 check 内累加用 int。

对策：mid 写 l + (r − l) / 2；凡是累加量一律 long long。

④ 方向反（收敛到区间端点）

症状：答案恒为 0 或上界。病灶：check 为真时应往哪边走想反了。

对策：拿样例手推两轮；把 check 的返回值命名成明确语义（如 stillOwe）。

拿到一道题，三问定方向

解题决策流程

第 1 问 答案是不是数组里已经存在的某个元素？ → **是 → 下标二分（T01 / T02）**

第 2 问 不是数组元素的话，能不能写出一个「越大越难满足」的判定？ → **能 → 答案二分**

第 3 问 · 答案是整数吗

是 → 整数域二分：l = mid ± 1，闭区间循环，ans 记录（T03 砍树）。

否 → 实数域二分：固定 100 轮，l = mid / r = mid（T04、T05）。

如果两问都答「不能」

说明这题不适合二分，或者单调性还没被你发现。先暴力过小数据，再想办法证明单调性 —— 不要硬套模板。

检查清单（提交前）

① 排序了吗 ② mid 写法安全吗 ③ 累加量够大吗 ④ check 方向对吗 ⑤ 边界点试过吗

为什么非要二分：两组数字说服你

数据规模 n	顺序查找 n 次	二分查找 $\lceil \log_2 n \rceil$ 次	快多少倍
100	100	7	约 14 倍
10,000	10,000	14	约 700 倍
1,000,000	1,000,000	20	约 5 万倍
1,000,000,000	10 亿	30	约 3300 万倍

对数增长有多慢

n 翻一倍，二分只多比较 1 次。从 100 万涨到 10 亿，规模涨了 1000 倍，比较次数只从 20 涨到 30。

代价：必须先排序

排序 O(n log n) 是一次性成本。只要查询次数多（如 T01 的 m 次询问），摊到每次上几乎可以忽略。

竞赛里的经验值

1 秒约能跑 10^8 次基本运算。看到 $n \leqslant 10^5$ 而暴力是 $O(n^2)$，就该条件反射想二分。

本课小结：二分是一把「有前提的刀」

第 5 课 · 分治算法（一）二分查找

1 **二分的前提是单调**
数组有序，或答案空间的判定具备「一边全真、一边全假」的结构。

2 **整数域：闭区间 + ±1 移动**
while (l ⩽ r)，mid = l + (r − l) / 2，可行就记 ans 再往目标侧挪。

3 **实数域：固定轮数，不写 eps**
100 轮足够精确到 double 极限；l = mid / r = mid 不需要 ±1。

4 **二分答案：难点全在 check**
框架永远是那五行，题目之间的差别只在判定条件与答案范围。

5 **三个隐性杀手：溢出、方向、边界**
二分题 90% 的失分不是「不会」，而是这三处细节。

一句话总纲

先证单调，再套模板；
边界靠手推，细节靠清单。

下节课预告

第 6 课进入分治的另一半：归并排序。

同样是「分而治之」，但分完之后要把两半合起来 —— 那一步才是真正的难点。

第 5 课 · 课后任务

把模板敲熟，把边界推明白

必做

T01 P2249 查找 —— 手写两遍，第二遍不看课件

T03 P1873 砍树 —— 写出 check 并手推样例

T05 P1163 银行贷款 —— 独立完成

选做

T02 P1102 A-B 数对 —— 先用哈希写一遍，再用二分写一遍

T04 P3743 小鸟的设备 —— 重点体会实数域写法

下节课 · 第 6 课

分治算法（二）：归并排序

预习问题：把两个已经排好序的数组，合成一个有序数组，最少要比较几次？

提交时间：本周五 22:00 前 · 洛谷提交后截图